Joblet

Process Isolation & Remote Execution Platform

System Design & Architecture Overview

Jay Ehsaniara

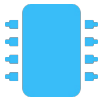
github.com/ehsaniara/joblet

What is Joblet?

A remote job execution system that runs arbitrary Linux commands in fully isolated, resource-controlled environments with real-time observability.

Kernel-level Isolation

Every job runs in its own PID, mount, IPC, UTS, and cgroup namespace , enforced by the Linux kernel, not application logic.



Resource Control

CPU, memory, I/O, GPU limits via cgroups v2. Jobs can't exceed their allocation or affect the host or each other.

Real-time Streaming

Live stdout/stderr streaming, historical log replay, and seamless reconnection mid-job , no data loss.

System Components

Client Layer

rnx CLI (Go)

Cross-platform binary for macOS, Windows, Linux

Python SDK

joblet-sdk-python for programmatic access

macOS (Silicon + Intel) · Windows · Linux desktop

gRPC + mTLS

Joblet Server (Linux)

Server Mode (Privileged)

gRPC listener, cgroup creation, namespace setup

Init Mode (PID 1)

Job A - unprivileged

Init Mode (PID 1)

Job B - unprivileged

cgroups v2 · namespaces · chroot · overlayFS

Ubuntu · RHEL · Amazon Linux

Two-Stage Execution Model

Same binary, two modes , security boundary enforced by the kernel

Server Mode (Privileged)

1. Receive gRPC request
2. Create cgroup + set limits
3. Allocate network IP
4. Re-exec as init mode

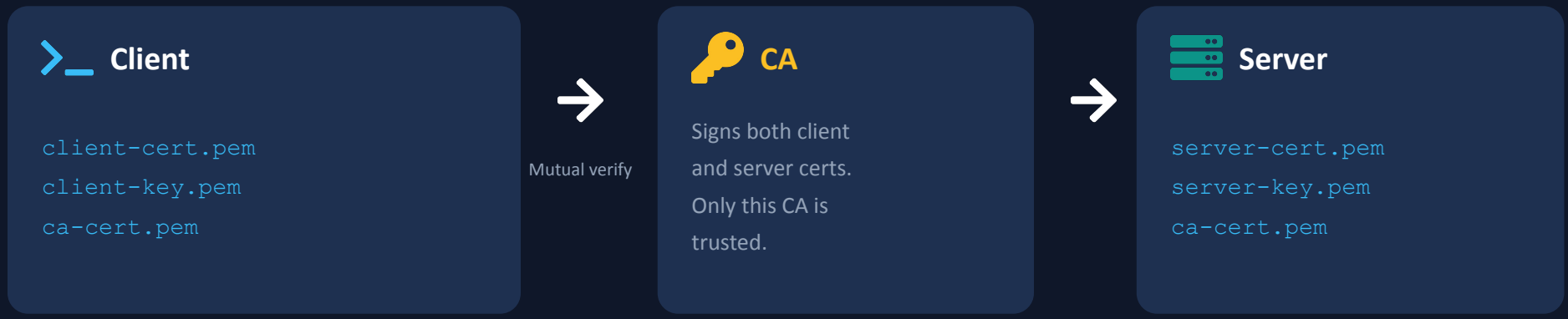
Privilege Boundary

Init Mode (Unprivileged)

5. Assign self to cgroup
6. Chroot + mount filesystem
7. Wait for network ready
8. exec(user command)

Security Architecture

Mutual TLS authentication with certificate-based role authorization



Role-Based Access (from certificate OU field)

Role	RunJob	StopJob	GetStatus	ListJobs	GetLogs	DeleteJob
OU=admin	✓	✓	✓	✓	✓	✓
OU=viewer	✗	✗	✓	✓	✓	✗

Resource Control , cgroups v2

Cgroup Hierarchy

```
/sys/fs/cgroup/
  └─ joblet.slice/
      └─ joblet.service/
          ├─ job-{uuid-A}/
          │    └─ proc/   ← PID here
          └─ job-{uuid-B}/
               └─ proc/   ← PID here
```

Enforced Limits

CPU	cpu.max - percentage or core pinning
Memory	memory.max - hard ceiling in bytes
I/O	io.max - bandwidth throttling
PIDs	pids.max - process count limit
GPU	Device nodes + namespace isolation

`Systemd: Delegate=yes · DelegateControllers=cpu memory io pids`

Namespace Isolation

Each job runs in 5 Linux namespaces , the kernel enforces boundaries

`CLONE_NEWPID`	**PID**	Job is PID 1 in its namespace. Can't see host or other job processes.
`CLONE_NEWNS`	**Mount**	Chroot jail with minimal filesystem. Host paths don't exist.
`CLONE_NEWIPC`	**IPC**	Separate shared memory, semaphores. No cross-job IPC.
`CLONE_NEWUTS`	**UTS**	Isolated hostname and domain name.

Network namespace is handled separately via the 2-phase network setup , not via CLONE_NEWNET. This enables bridge, isolated, custom, and none modes.

Network Isolation Modes

Bridge (default)

NAT to host, internet access.
DHCP IP from 172.20.0.0/16.
Jobs on same bridge can talk.

```
--network=bridge
```

Custom

User-defined CIDR range.
Own bridge per network.
Inter-job comms within network.

```
--network=myapp
```

Isolated

Point-to-point veth.
Outbound internet only.
No inter-job communication.

```
--network=isolated
```

None

Loopback only.
Zero network access.
Maximum security.

```
--network=none
```

Storage & Runtime Environments

Persistent Volumes

- **Filesystem**
  Persistent across jobs and reboots
- **Memory (tmpfs)**
  Ultra-fast I/O, cleared on completion
- **Isolation**
  Kernel-level: bind mounts in chroot
- **Sharing**
  Same volume mountable by multiple jobs

```
rnx volume create data --size=1GB
```

Runtime Environments

Pre-built execution environments via OverlayFS:

```
Host root /   →  Lower (read-only)
Temp dir      →  Upper (captures writes)
Merged view →  Chroot for installs
Runtime dir ←  Copy binaries only
```

14-phase build pipeline with real-time streaming

Zero host contamination guaranteed

```
rnx runtime build ./runtime.yaml
```

Runtime Build , 14-Phase Pipeline

Each phase streams progress events back to the client in real-time

1 Validate

1. Parse & validate YAML
2. Detect platform & arch
3. Check disk space
4. Validate packages

2 Build (OverlayFS)

5. Prepare directories
6. Pre-install hook
7. Install base packages
8. Install pip/npm pkgs
9. Post-install hook

3 Package & Validate

10. Copy binaries from overlay
11. Copy libraries from overlay
12. Copy configuration files
13. Generate runtime.yml
14. Validate build integrity

OverlayFS Isolation: Host root (/) = read-only lower → Temp dir = upper (captures writes) → Merged = chroot for installs → **Copy binaries only.**

Runtime Specification & Languages

runtime.yaml

```yaml
schema_version: "1.0"
name: python-3.11-ml
version: 1.0.0
description: Python 3.11 with ML

base:
  language: python
  version: "3.11"

pip:
  - numpy
  - pandas
  - scikit-learn

libraries:          # System libs to copy
  - libopenblas*

environment:
  PYTHONUNBUFFERED: "1"

hooks:
  pre_install: |
    apt-get install -y libopenblas-dev
```

Supported Languages

Python	3.9, 3.10, 3.11, 3.12	`pip`
Java	11, 17, 21	`maven`
Node.js	18, 20	`npm`
Go	1.21+	`go mod`
Rust	1.75+	`cargo`

On-disk Structure

```
/opt/joblet/runtimes/
  └─ python-3.11-ml/
       ├─ runtime.yml
       └─ isolated/
            ├─ bin/ lib/ usr/
            └─ etc/  (self-contained)
```

Without runtime: apt-get && pip install ... 15-30 min **With runtime:** --runtime=python-3.11-ml 2-3 sec

Real-Time Log Streaming

One API call handles live streaming, historical replay, and seamless reconnection

Running Job

gRPC server-streaming RPC pushes DataChunk messages to the client as stdout/stderr arrives. Real-time, low-latency.

Completed Job

Persist service returns historical log data. Client sees the full output as if it was there from the start.

Reconnecting

Fetches historical logs first (everything missed), then transparently switches to live streaming. Zero data loss.

```
rnx job log <job-id>
```

Shows all logs from beginning to current point , running or finished

CLI in Action , rnx job run

Simple execution

```
rnx job run echo "Hello, World!"
```

Resource limits

```
rnx job run --max-cpu=200 --max-memory=4096 \
  python3 train.py
```

Full-featured

```
rnx job run \
  --max-cpu=400 --max-memory=8192 \
  --gpu=1 --gpu-memory=8GB \
  --network=mynet \
  --volume=data --runtime=python-3.11-ml \
  --upload=./src --env=EPOCHS=100 \
  --secret-env=API_KEY=sk-... \
  python3 train.py
```

Scheduled

```
rnx job run --schedule="2h30m" \
  --timeout=5m backup.sh
```

Key Takeaways


Two-stage execution separates privileged setup from unprivileged job execution


mTLS with certificate-based RBAC , no insecure mode, no system CA trust


Kernel-enforced isolation via 5 namespaces + cgroups v2 resource limits


4 network modes from full access (bridge) to zero access (none)


Smart log streaming handles live, historical, and reconnection transparently


OverlayFS runtime builds guarantee zero host contamination


Single Go binary for both server and init mode , version consistency

github.com/ehsaniara/joblet